

INDEX

INDEX

1. Playwright (JavaScript)

- Core Concepts
 1. Playwright Architecture
 2. Installation & Project Setup
 3. Test Runner (test, expect)
 4. Browser Context & Page
- Locators
 1. locator()
 2. getByRole()
 3. getByText()
 4. CSS Selectors
 5. XPath
 6. Best Practices for Locators
- Actions & Assertions
 1. click(), fill(), type()
 2. Assertions (toHaveText, toHaveURL)
 3. Handling waits (auto-waiting)
- Framework Design
 1. Page Object Model (POM)
 2. Test structure organization
 3. Fixtures
 4. Reusable components
- Advanced Topics
 1. Handling Frames
 2. Handling Multiple Tabs
 3. Network Interception
 4. API mocking
 5. Debugging (trace viewer, pause)
 6. Parallel execution
 7. Config file ([playwright.config.js](#))

- Additional Playwright Topics (Post Advanced Section)
 1. Test Data Handling (JSON / Fixtures)
 2. Error Handling & Retry Strategy
 3. CI/CD Integration (Jenkins Pipeline Basics)
 4. Reporting (Allure / HTML Reports)
 5. Environment Configuration (Multiple Envs - Dev/QA/Prod)
 6. Test Execution Strategies (Headless / Headed / Parallel in CI)

2. API Automation

- Basics
 1. HTTP Methods (GET, POST, PUT, DELETE)
 2. Status Codes
 3. Headers & Parameters
- Validation
 1. Response validation
 2. JSON parsing
 3. Schema validation
- Advanced
 1. Authentication (OAuth, Tokens)
 2. API chaining
 3. Error handling
 4. Performance basics

3. Java (Interview Level)

- Core
 1. OOP Concepts (Encapsulation, Inheritance, Polymorphism)
 2. Classes & Objects
 3. Constructors
- Collections
 1. List, Set, Map
 2. ArrayList, HashMap
 3. Iteration techniques

- Problem Solving
 1. Strings manipulation
 2. Loops
 3. Basic algorithms
- Exception Handling
 1. Try-catch
 2. Custom exceptions

4. Automation Framework Design

- 1 TestNG structure
- 2 Page Object Model
- 3 Utilities & Helpers
- 4 Data-driven testing
- 5 Reporting (Allure, Extent)
- 6 CI/CD integration (Jenkins)

5. DSA

- 1 Arrays
- 2 Strings
- 3 HashMap
- 4 Two Pointers
- 5 Sliding Window
- 6 Stack
- 7 Recursion
- 8 Trees & Graphs (basic)

6. SQL (Database)

- 1 Basic queries (SELECT, WHERE)
- 2 Joins (INNER, LEFT)
- 3 Group By, Order By
- 4 Data validation queries

7. 🧠 NEW SECTION: AI FOR TESTING

Topics you must cover:

1. What is Prompt Engineering
2. How AI generates test cases
3. AI vs deterministic testing
4. What is agentic workflow
5. How to test AI responses

CORE CONCEPTS

PLAYWRIGHT → CORE CONCEPTS (COMPLETE)

We will cover:

1. Architecture
 2. Installation & Project Setup
 3. Test Runner (test, expect)
 4. Browser Context & Page
-

1 PLAYWRIGHT ARCHITECTURE

1 What is it

Playwright architecture defines how tests interact with browsers using a **Node.js layer + browser engines (Chromium, Firefox, WebKit)**.

2 Key Components

- Test Script (JS/TS)
 - Playwright API
 - Browser Server
 - Browser Engines
 - Browser Context
 - Page
-

3 How YOU should answer (Interview Style)

Playwright follows a client-server architecture where test scripts communicate with browser engines through Playwright APIs. It supports multiple browsers like Chromium, Firefox, and WebKit, and uses browser contexts for isolated test execution.

4 Practical Example

```
</> JavaScript   
  
test('example', async ({ page }) => {  
  await page.goto('https://example.com');  
});
```

👉 Script → Playwright → Browser → Execution

5 Common Mistakes

- ✗ Saying “Playwright only works on Chrome”
 - ✗ Not mentioning **browser context**
 - ✗ Over-explaining internal working
-

6 Expected Questions

- What is Playwright architecture?
 - How is it different from Selenium?
 - What is browser context?
-

2 INSTALLATION & PROJECT SETUP

1 What is it

Setting up Playwright project using Node.js and installing required dependencies.

2 Key Components

- Node.js
 - npm
 - @playwright/test
 - Browsers installation
 - Project structure
-

3 How YOU should answer

I set up Playwright using Node.js by installing @playwright/test via npm. Then I install browser binaries and create a test folder structure for writing test cases.

4 Practical Example

```
</> Bash
npm init -y
npm install -D @playwright/test
npx playwright install
```



5 Common Mistakes

- ✗ Saying "Playwright needs Selenium"
- ✗ Not knowing commands
- ✗ Not knowing project structure

6 Expected Questions

- How do you install Playwright?
 - What are prerequisites?
 - What is project structure?
-

3 TEST RUNNER (test, expect)

1 What is it

Playwright Test Runner is used to execute tests and perform assertions.

2 Key Components

- `test()`
 - `expect()`
 - hooks (`beforeEach`, `afterEach`)
 - fixtures
-

3 How YOU should answer

Playwright provides a built-in test runner where `test()` defines test cases and `expect()` is used for assertions. It supports fixtures and hooks for better test management.

4 Practical Example

```
JavaScript

test('title check', async ({ page }) => {
  await page.goto('https://example.com');
  await expect(page).toHaveTitle(/Example/);
});
```

5 Common Mistakes

- ✗ Confusing test() with function
- ✗ Not knowing expect() usage
- ✗ Ignoring hooks

6 Expected Questions

- What is test() in Playwright?
- What is expect()?
- What are hooks?

4 BROWSER CONTEXT & PAGE

1 What is it

Browser context is an isolated session, and page represents a tab.

2 Key Components

- Browser
- Context
- Page

3 How YOU should answer

Browser context is like a separate session similar to incognito mode, and page represents a single tab. This helps in running tests independently without affecting each other.

4 Practical Example

```
</> JavaScript  
  
const context = await browser.newContext();  
const page = await context.newPage();
```

5 Common Mistakes

- ✗ Saying context = browser
 - ✗ Not understanding isolation
 - ✗ Confusing page and context
-

6 Expected Questions

- What is browser context?
- Difference between browser and context?
- Why is context important?

LOCATORS



PLAYWRIGHT → LOCATORS (PART 1)

We'll cover in parts:

1. locator()
 2. getByRole() ★
 3. getByText()
 4. CSS & XPath
 5. Best Practices
-



1 locator()

1 What is it

`locator()` is a Playwright method used to find elements using selectors like CSS, XPath, id, etc.

2 Key Components

- CSS selectors (`#id`, `.class`)
 - XPath
 - Chaining (`locator().locator()`)
 - Auto-waiting support
-

3 How YOU should answer (Interview Style)

`locator()` is a flexible method in Playwright used to identify elements using CSS or XPath selectors. It supports auto-waiting and allows chaining for better element targeting.

4 Practical Example

```
</> JavaScript   
  
await page.locator('#username').fill('Kalpesh');  
await page.locator('.login-btn').click();
```

5 Common Mistakes

- ✗ Using long XPath unnecessarily
 - ✗ Not using chaining
 - ✗ Over-relying on CSS when better options exist
-

6 Expected Interview Questions

- What is locator() in Playwright?
 - Difference between locator() and getByRole()?
 - Why is locator better than findElement (Selenium)?
-

2 getByRole() (MOST IMPORTANT)

1 What is it

Used to locate elements based on **ARIA roles (accessibility-based)**.

2 Key Components

- Role (button, textbox, link)
 - Name (visible label/text)
 - Accessibility tree
-

3 How YOU should answer

getByRole() is the most recommended locator in Playwright as it uses accessibility roles, making tests more stable, readable, and aligned with real user interactions.

4 Practical Example

 JavaScript

```
await page.getByRole('button', { name: 'Login' }).click();
```



5 Common Mistakes

-  Not using role-based locators
 -  Ignoring accessibility
 -  Using XPath instead
-

6 Expected Interview Questions

- Why is getByRole preferred?
 - What is the ARIA role?
 - Difference between getByRole and locator?
-

3 getText()

1 What is it

Used to locate elements using **visible text on the page**.

2 Key Components

- Exact text match
 - Partial text match
 - Case sensitivity
 - Works on visible UI text
-

3 How YOU should answer (Interview Style)

getText() is used to locate elements based on visible text. It is simple and readable but less stable compared to role-based locators since UI text can change frequently.

4 Practical Example

 JavaScript

```
await page.getByText('Login').click();  
await page.getByText('Submit').click();
```



5 Common Mistakes

- ✗ Overusing text-based locators
- ✗ Not handling dynamic text
- ✗ Using when role-based locator is better

6 Expected Interview Questions

- When do you use getText()?
 - What are limitations of text locators?
-

4 CSS & XPath

1 What is it

Traditional selector strategies used to locate elements in DOM.

2 Key Components

- CSS (`#id`, `.class`, `tag`)
 - XPath (`//`, `contains()`, `text()`)
-

3 How YOU should answer (Interview Style)

CSS and XPath are fallback locator strategies. CSS is generally faster and preferred, while XPath is useful for complex DOM traversal when other locators are not sufficient.

4 Practical Example

 JavaScript

```
await page.locator('#username').fill('test'); // CSS
await page.locator('//button[text()='Login"]').click(); // XPath
```

5 Common Mistakes

- ✗ Writing long and complex XPath
 - ✗ Using XPath when better locators exist
 - ✗ Not optimizing selectors
-

6 Expected Interview Questions

- CSS vs XPath?
 - Which is faster and why?
 - When do you use XPath?
-

5 Best Practices (INTERVIEW GOLD)

1 What is it

Guidelines to write **stable, maintainable locators**.

2 Key Components

- Prefer getByRole()
 - Use data-testid
 - Avoid XPath
 - Keep selectors simple
 - Use chaining
-

3 How YOU should answer (Interview Style)

My approach is to prefer role-based locators like getByRole for stability. If not available, I use data-testid or CSS selectors. I avoid XPath unless necessary and focus on maintainable and readable locators.

4 Practical Example

</> JavaScript



```
await page.getByRole('button', { name: 'Submit' }).click();  
await page.locator('[data-testid="login-btn"]').click();
```

5 Common Mistakes

- ✗ Using absolute XPath
- ✗ Writing brittle selectors
- ✗ Ignoring readability

6 Expected Interview Questions

- What is your locator strategy?
- Which locator do you prefer and why?
- How do you write stable locators?

Actions & Assertions

PLAYWRIGHT → Actions & Assertions

This section is heavily asked because interviewers want to know:

- 👉 Can you actually interact with UI properly?
- 👉 Do you know validations?

We'll cover:

1. Actions
 2. Assertions
 3. Auto-waiting behavior
 4. Best Practices
-

1 Actions

1 What is it

Actions are methods used to interact with web elements like clicking, typing, selecting, hovering, etc.

2 Key Components

- `click()`
 - `fill()`
 - `type()`
 - `check()`
 - `uncheck()`
 - `hover()`
 - `selectOption()`
 - `press()`
-

3 How YOU should answer (Interview Style)

Actions in Playwright are used to simulate real user interactions such as clicking buttons, entering text, selecting dropdown values, and hovering over elements. The playwright automatically waits for elements before performing actions.

4 Practical Example

```
</> JavaScript
await page.locator('#username').fill('Kalpesh');
await page.getByRole('button', { name: 'Login' }).click();
await page.locator('#country').selectOption('India');
```

5 Common Mistakes

- ✗ Using unnecessary waits before actions
 - ✗ Using force click without reason
 - ✗ Confusing fill() and type()
-

6 Expected Interview Questions

- Difference between fill() and type()?
 - What actions have you used in Playwright?
 - How does Playwright handle waits during actions?
-

2 Assertions

1 What is it

Assertions are validations used to verify expected behavior or UI state.

2 Key Components

- toHaveText()
 - toBeVisible()
 - toHaveURL()
 - toHaveTitle()
 - toBeChecked()
-

3 How YOU should answer (Interview Style)

Assertions in Playwright are used to validate application behavior. Playwright provides built-in auto-retrying assertions which improve test stability and reduce flaky tests.

4 Practical Example

```
</> JavaScript
await expect(page).toHaveTitle(/Dashboard/);
await expect(page.locator('.success-msg')).toBeVisible();
await expect(page.locator('#name')).toHaveValue('Kalpesh');
```

5 Common Mistakes

- ✗ Using manual validations instead of expect()
- ✗ Not understanding auto-retry behavior
- ✗ Writing weak assertions

6 Expected Interview Questions

- What is assertion in Playwright?
 - Difference between hard and soft assertion?
 - Why are Playwright assertions stable?
-

3 Auto-Waiting (VERY IMPORTANT)

1 What is it

The playwright automatically waits for elements to become ready before performing actions or assertions.

2 Key Components

- Visibility check
 - Element enabled state
 - Stability check
 - Auto-retrying assertions
-

3 How YOU should answer (Interview Style)

One major advantage of Playwright is built-in auto-waiting. Before performing actions, Playwright ensures elements are visible, stable, and actionable, reducing flaky tests and minimizing explicit waits.

4 Practical Example

 JavaScript

```
await page.getByRole('button', { name: 'Submit' }).click();
```



 No explicit wait required.

5 Common Mistakes

-  Adding unnecessary `waitForTimeout()`
 -  Using hard waits everywhere
 -  Not trusting Playwright auto-wait
-

6 Expected Interview Questions

- What is auto-waiting in Playwright?
 - Difference between Playwright and Selenium waits?
 - Why does Playwright reduce flaky tests?
-

4 Best Practices

1 What is it

Guidelines for writing reliable actions and assertions.

2 Key Components

- Prefer built-in assertions
- Avoid hard waits
- Use stable locators
- Validate meaningful UI behavior

3 How YOU should answer (Interview Style)

I avoid hard waits and rely on Playwright's built-in auto-waiting and assertions. I also use stable locators and meaningful validations to improve test reliability.

4 Practical Example

JavaScript

```
await expect(page.getByRole('heading')).toHaveText('Dashboard');
```

5 Common Mistakes

- ✗ Using `waitForTimeout()` frequently
 - ✗ Weak validations
 - ✗ Over-validating unnecessary UI elements
-

6 Expected Interview Questions

- What are best practices for assertions?
- How do you avoid flaky tests?
- Why avoid hard waits?